

11. Paralelno procesiranje

Kada govorimo o paralelnoj obradi podataka i izvršavanju zadataka u računarskim sistemima, potrebno je da shvatimo zašto i kako smo došli u tu situaciju. Neka postoji računar sa 2 GB RAM koji treba da ikopira fajl veličine 1 GB na eksterni disk. Program koji kopira fajl raspolaže sa dovoljno memorije da kompletan fajl učitava u memoriju i zatim isti sačuva u određeni direktorijum. Ako je veličina fajla 10 GB, takav način rada nije moguć jer nema dovoljno slobodne radne memorije. Klasično rješenje je dijeljenje fajla u manje dijelove, recimo 1 MB, te kopiranje dio po dio dok ne iskopiramo cijeli fajl. Ovakav pristup nam nudi brojne pogodnosti, od manjeg zauzeća memorije, preko činjenice da ne moramo da čekamo da se kompletan fajl učitava u memoriju prije nego što ga počnemo snimati, do oslobađanja procesora za druge zadatke dok čeka da se neki bafer iskopira na disk (jer koristimo DMA, itd). Da bi ostvarili ove prednosti, obično nam je neophodna podrška za izvršavanje više simultanih zadataka (taskova) od strane operativnog sistema i takve operativne sisteme nazivamo **multitasking operativni sistemi**.

Ukoliko je u toku kopiranja potrebno i vršiti neku transformaciju podataka koja je procesorski intenzivna (npr. enkripcija) i ukoliko sistem posjeduje više procesora ili jezgara, moguće je ostvariti bolje performanse tako što će svako jezgro dobiti svoj dio posla čime vršimo **paralelizaciju programa**. Kao što je rečeno, da bi zaista i ostvarili bolje performanse, neophodno je postojanje nekog vida paralelizma i saradnje među dijelovima zadatka, pri čemu paralelne mašine grubo možemo podijeliti na **višeprocesorske mašine** i **klastere**, a način saradnje može biti zasnovan na **dijeljenoj memoriji** i **komunikaciji porukama**.

Kada govorimo o multiprocesorskim mašinama, postoji više načina ostvarivanja paralelizma unutar jednog računara. Na najnižem nivou je moguće ostvarivanje paralelizma na nivou instrukcije ili unutar samog procesorskog jezgra (SIMD, MIMD, SPMD), na nivou fiktivnog jezgra (Hyper-Threading), na nivou višestrukih jezgara unutar jednog procesora ili postojanja više CPU-ova u jednom računaru. Kod postojanja velikog broja procesora i jezgara unutar jednog računara moramo da se izborimo sa problemom simultanog pristupa dijeljenim resursima, od kojih je najznačajnija radna memorija. Kako je sama memorija sporija od procesora (što rezultuje upotrebom prilično kompleksnih keš struktura u procesorima), očigledno je da će u istovremene upotrebe od strane više procesora taj problem biti još izraženiji. Takođe, doći ćemo u situaciju u kojoj nije svejedno kojem dijelu memorije pristupamo iz kog procesora što se naziva NUMA (Non-Uniform Memory Access) i za čije rješavanje je potrebna i podrška operativnog sistema (kako bi pravilno izvršio i raspoređivanje zadataka na odgovarajuće procesore i alocirao memoriju u "najbližim" regionima) ali i smislen odabir algoritma i podjele podataka između taskova (jer ne želimo da pristupamo "tuđoj" memoriji bez prijeke potrebe).

Klasteri predstavljaju skup povezanih računara (čvorova), pri čemu svaki od čvorova klastera izvršava sopstveni operativni sistem i posjeduje sopstvene resurse, procesorske i memorijske, a tipično su međusobno povezani upotrebom neke mrežne tehnologije. Postoje sistemi koji se zaista sastoje od većeg broja fizički odvojenih računara, ali logički predstavljaju jednu cjelinu i izvršavaju istu sliku operativnog sistema (uz ozbiljnu potrebu za pažnju pri radu sa NUMA jer je pristup *izuzetno* neuniforman), ali takve sisteme nazivamo **masivno višeprocesorski sistemi** i nećemo ih posmatrati kao klastere za potrebe ovog predmeta. Razlozi za upotrebu klastera se mogu, na najvišem nivou, podijeliti u dvije kategorije: poboljšanje stabilnosti i poboljšanje performansi.

Saradnja upotrebom dijeljene memorije se obično realizuje kroz upotrebu više niti (jer su efikasije u ovom slučaju od višestrukih procesa). Ovaj pristup je moguće realizovati na mnoštvo načina, a jedan vrlo često korišćen (i jednostavan za upotrebu) je i upotreba **OpenMP** standarda. Kod komunikacije porukama takođe postoji mnoštvo mogućih rješenja, ali se u high-performance computing oblasti najčešće koristi **MPI** (Message Passing Interface) standard za komunikaciju koji se može koristiti i unutar jednog čvora (što je manje efikasno od upotrebe dijeljene memorije jer je memorija najbrži komunikacioni medij) i između čvorova klastera. Naravno, često se koristi hibridni pristup koji nam omogućava da unutar čvora koristimo OpenMP, a između čvorova MPI. Na ovaj način ostvarujemo dobre performanse unutar svakog čvora, ali više nismo ograničeni resursima samo jednog čvora (npr. za neku simulaciju nam je potrebno 480 GB memorije, a svaki od čvorova posjeduje samo 64 GB - simulaciju možemo izvršiti na klasteru sa osam čvorova i ukupno 512 GB memorije).

Kako su klasteri često vrlo skupi i veliki broj korisnika želi da ih koristi, potrebno je definisati i način na koji pristupate klasteru, kada je koja simulacija na redu za izvršavanje, itd. Kako bi ovaj posao brzo postao zamoran za čovjeka, a da ne govorimo o tome da je vrlo teško ostati objektivan i smiren u okruženju u kojem se svi bore za vrijeme na superračunaru, u praktično svim slučajevima se koriste sistemi koji se zovu **JMS (Job Management Systems)**. Zadatak ovih sistema je da na efikasan i predvidiv način upravljaju resursima, listama čekanja za poslove, ali i samim raspoređivanjem procesa na čvorove klastera, te upravljenje fajlovima koji su potrebni za poslove. Postoji više konkretnih sistema koji se koriste, od kojih ćemo izdvojiti nekoliko najpopularnijih: Condor, PBS (Portable Batch System) razvijen od strane NASA, Torque (otvoreni kod, kompatibilan sa PBS) i Maui (novije i naprednije rješenje, pogodniji za nove generacije HPC klastera).

12. Vrste klastera

Failover klasteri omogućavaju poboljšanje stabilnosti sistema upotrebom višestrukih komponenata, pri čemu su komponente čvorovi klastera. Najjednostavniji failover klaster se sastoji od dva identična čvora koji obavljaju isti posao tako da u slučaju otkaza primarnog čvora sekundarni čvor nastavlja da opslužuje korisnike. Naravno, ukoliko se klaster sastoji od dva čvora, i dođe do različitog rada čvorova (jer je jedan čvor neispravan), često je teško odrediti koji čvor je u pravu, a koji nije (jer oba misle da jesu). Obično se ovakvi problemi rješavaju upotrebom neparnog broja čvorova i postojanjem **kvoruma** (kvalifikovane većine) jer sada možemo da vjerujemo da ako su dva čvora suprotstavljena jednom, vrlo vjerovatno su dva čvora u pravu.

Load-balancing klasteri predstavljaju klastere koji su zasnovani na ideji raspodjele opterećenja na čvorove klastera sa ciljem poboljšanja performansi. Sami čvorovi ne moraju biti homogeni i moguće je postojanje čvorova različitih performansi (npr. da bi klaster mogao obraditi 800 zahtjeva u sekundi, moguće je napraviti klaster od dva čvora koji mogu da obrade po 400, ali i klaster sa čvorovima koji mogu da obrade 300, 200, 150, 100 i 50 zahtjeva u sekundi). Ovaj pristup je vrlo prisutan u upotrebi u internet servisima jer je sam HTTP pogodan za takvu upotrebu (nema stanja, nema dugoživućih konekcija), a čak i samo upotrebom DNS-a možemo obezbijediti primitivno round-robin raspoređivanje (jer je za jedno simboličko ime moguće postaviti više IP adresa koje će biti servirane klijentima po round-robin pristupu). Alternativno, ispred samih servera se nalazi balanser opterećenja kao poseban mrežni uređaj koji je zadužen za nadzor rada servera i pravilno raspoređivanje upita na njih. Najveći problem kod ovog pristupa leži u činjenici da ukoliko dođe do pada balansera nijedan server više nije dostupan. Da bi to izbjegli, moguće je napraviti balansere u fail-over konfiguraciji.

High-performance klasteri su klasteri koji su napravljeni sa idejom da korisnicima pruže visoke performanse, čak i po cijenu da su i organizacija klastera i tehnologije koje koristimo vrlo specifične za tačno određenu namjenu. Ova klasa je vrlo široka i može se podijeliti na tri osnovne podklase: custom-made, beowulf i COW. **Custom-made** klasteri su namjenski pravljeni po narudžbi klijenata i često uključuju komponente ili sklopove koji su razvijeni samo za tu potrebu (npr. akceleratorске kartice za simulaciju nuklearnih interakcije ili "rudarenje" kriptovaluta). Kako obezbjeđuju vrhunske performanse i cijena im je obično vrhunska. Sami čvorovi koji izvršavaju korisničke poslove mogu imati posebne i namjenski razvijene operativne sisteme, dok se za front-end čvor namijenjen za interakciju sa korisnicima obično koristi neki od derivata Linux operativnog sistema (jer možemo u potpunosti da ga prilagodimo zahtjevima). **Beowulf** klaster se sastoji od klasičnih računara, ali u minimalnoj konfiguraciji. To znači da svaki čvor sadrži samo minimum potrebnih komponenti (bez periferija, bez grafičkih kartica ako nisu GPGPU, ali često i bez sekundarne memorije jer je koriste putem mrežnih servisa sa nekom centralnog servisa). Na taj način je smanjena cijena sistema, ali i potrošnja električne energije jer se ispostavlja da je kod klastera visokih performansi to izuzetno izražen ekonomski i toplotni problem. **COW (cluster of workstations)** je sličan koncept kao Beowulf, ali su sada čvorovi potpune radne stanice i mogu se koristiti i van klastera (sadrže periferije, itd). Ovakvi klasteri su obično *ad-hoc* ili privremeni klasteri (npr. računarska laboratorija koja se noću koristi za simulacije). U oba slučaja se koriste komponente koje je moguće pronaći kod klasičnih prodavaca računara (Commercial Off-The Shelf ili COTS), a ne samo kod specijalizovanih firmi koje se bave klasterima. Često se dopušta odstupanje u mrežnom segmentu u kojem se koriste nešto naprednije tehnologije (npr. 10G Ethernet

umjesto Gigabit Ethernet, itd). Ovo je važan aspekt jer nam mrežna infrastruktura često direktno diktira skalabilnost sistema i broj čvorova koji možemo efikasno da koristimo.

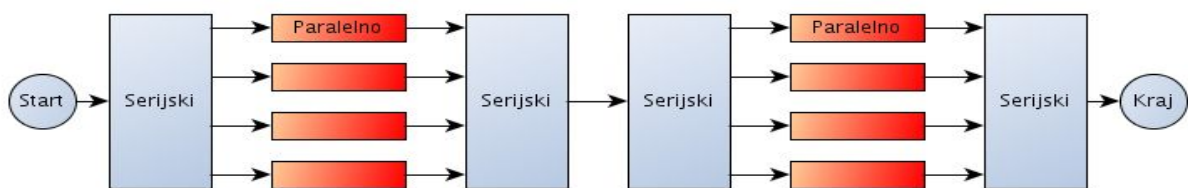
Kada govorimo o mrežnom povezivanju, kod HPC klastera se teži bržim linkovima (bandwidth), ali i manjem kašnjenju (latency) tako da su upotrebi često 40G Ethernet ili Infiniband tehnologije, pri čemu je Infiniband trenutno doimnantna tehnologija za veće HPC infrastrukture. U zavisnosti od ciljane upotrebe klastera vršimo i projektovanje topologije mreže koja može biti jednostavna zvijezda (ako centralni uređaj ima dovoljan broj portova), stablo (ako nam je prihvatljiv određeni pad performansi), ali i namjenska topologija (grid, hypercube, hypertorus, itd). Na primjer, kod klastera koji se koriste za simulaciju vremenskih prilika, sam domen je izdijeljen i samo susjedni dijelovi utiču na bilo koji dio, tako da se često mreža realizuje na taj način da je svaki čvor direktno povezan na "susjede" jer su oni jedini sa kojima i komunicira u toku simulacije.

Poseban problem predstavlja činjenica da ne postoje jasne granice između navedenih klasa klastera. Očigledno je da se Beowulf i COW klasteri razlikuju u detaljima, ali i da je zadatak load-balancing klastera vrlo sličan high-performance klastera - bolje performanse. Ono što izdvaja HPC od LBC je činjenica da HPC obično koriste vrlo napredne (i skupe) mrežne tehnologije kako bi obezbijedili dobre performanse i kod snažno spregnutih problema (kod kojih postoji mnogo komunikacije između servera) dok su LBC pogodniji za slabo spregnute ili probleme koje je moguće trivijalno paralelizovati (jer ima malo ili nimalo komunikacije između čvorova u toku obrade). Takođe je važno primijetiti da LBC koji ima više resursa nego što mu je potrebno (npr. za 800 z/s posjeduje pet servera po 200 z/s ili ukupno 1000 z/s) može nastaviti funkcionisati i u slučaju otkaza jednog čvora što ga čini i fail-over klasterom.

13. OpenMP

OpenMP (Open Multi-Processing) je standardizovani API koji pojednostavljuje rad sa nitima tako što ih automatski ih pravi i terminira, vrši raspodjelu opterećenja i sinhronizaciju niti, te na taj način olakšava paralelizaciju rada (rad sa privatnim i dijeljenim varijablama, jednostavne paralelne petlje, itd). Važno ograničenje OpenMP je da riječ o pristupu koji je moguće koristiti samo u mašinama sa dijeljenom memorijom, bez komunikacije porukama i distribuiranih sistema.

OpenMP program koji koristi paralelizam prisutan u mašini funkcioniše na sljedeći način: serijski kod se izvršava na klasičan način, ali u određenim trenucima dijelimo izvršavanje na više niti koje se izvršavaju simultano. Ideja je da nije potrebno sve zadatke izvršavati paralelno (npr. pokretanjem većeg broja procesa ili niti pri samom početku izvršavanja) nego se paralelizam ostvaruje u bitnim sekcijama koda čija paralelizacija može donijeti poboljšanje performansi.



Za C i C++ koristimo direktive kompajleru **#pragma omp** i opcione vitičaste zagrade za blok na koji se odnosi paralelna sekcija. Primjer koda prvog OpenMP programa:

```
#include <omp.h>
#include <stdio.h>

int main (int argc, char **argv) {
    #pragma omp parallel
    {
        printf ("Moj prvi OpenMP program!\n");
    }
}
```

Sada možemo da sačuvamo fajl pod nazivom `omp1.c` i isti kompajliramo:

```
gcc -fopenmp omp1.c -o omp1
```

Pokretanje se vrši kao da pokrećemo običan izvršni fajl:

```
./omp1
Moj prvi OpenMP program! - 4, 8 ili 16 puta
```

Program će biti pokrenut u jednom procesu, ali će dio unutar **#pragma omp parallel { }** bloka biti izvršen u osam paralelnih niti (ako je CPU sa 8 jezgara ili dva CPU-a sa po 4 jezgra ili 8 CPU-ova sa po jednim jezgrom). Sam broj niti se najlakše kontroliše kroz sistemsku promjenjivu `OMP_NUM_THREADS` koja sadrži broj niti koji želimo, na primjer, ako želimo da koristimo tačno 4 niti:

```
export OMP_NUM_THREADS=4
./omp1
Moj prvi MPI program! - 4 puta
```

Isti efekat je moguće ostvariti i definisanjem i pozivanjem u istoj liniji:

```
OMP_NUM_THREADS=16 ./omp1
Moj prvi MPI program! - 16 puta
```

Alternativno, ako želimo da unutar koda programa izvršimo definisanje broja niti koje želimo da pokrenemo za neku sekciju (npr. znamo da se određeni dio koda najbolje izvršava sa četiri niti) upotrebom **omp_set_num_threads()** poziva.

Primjer jednostavnog programa koji vrši paralelno sumiranje elemenata matrice:

```
#define N 256
#include <stdio.h>
#include <omp.h>
int main (int argc, char **argv) {
    double matrica[N][N], suma=0.0;  int i,j;
    for(i=0;i<N;i++) {
        for(j=0;j<N;j++) { matrica[i][j]=i*N+j;
            printf("%12.2f ",matrica[i][j]); }
        printf("\n"); }
    #pragma omp parallel for reduction(+:suma) private(j)
    for(i=0;i<N;i++)
        for(j=0;j<N;j++)
            suma+=matrica[i][j];
    printf("Suma elemenata matrice je: %12.2f\n",suma);
}
```

Dio od interesa je linija

```
#pragma omp parallel for reduction(+:suma) private(j)
```

koja definiše da se petlja koja slijedi paralelizuje (čime je promjenjiva **i** automatski privatna za svaku nit), da se promjenjiva **j** posmatra kao privatna za svaku nit (jer će unutrašnja petlja biti nezavisna u svakoj niti), ali i da se vrši redukcija sumiranjem parcijalnih suma izračunatih u svakoj od niti u finalnu sumu. Raspodjela iteracija se vrši statički i praktično svaka nit dobija po 1/niti iteracija u kontinuitetu (za dvije niti bi prva nit dobila prvih 128, a druga drugih 128 iteracija).

Sama strategija raspodjele iteracija se može definisati na sljedeći način:

```
#pragma omp parallel for reduction(+:suma) schedule(strategija)
```

- **schedule(static, chunk-size)** - statički raspoređivač, dijeli petlju na dijelove veličine do chunk-size (default je približno iteracija/niti) i unaprijed raspoređuje posao po nitima sukcesivno
- **schedule(dynamic, chunk-size)** - dinamički raspoređivač, dijeli petlju (default chunk-size je 1!) i dinamički (i stohastički) dodjeljuje posao niti koja je trenutno slobodna
- **schedule(guided, chunk-size)** - dinamički raspoređivač, dijeli preostale iteracije po broju niti, chunk-size je minimalna preporučena veličina završnih iteracija
- **schedule(auto, chunk-size)** - kompajler bira najbolji
- **schedule(runtime, chunk-size)** - runtime definiše

Strategiju je moguće odrediti i iz komandne linije definisanjem **OMP_SCHEDULE** ili iz koda pozivom **omp_set_schedule(strategija)**.

14. Message Passing Interface - MPI

U situacijama u kojima nije moguće koristiti dijeljenu memoriju (npr. u distribuiranim sistemima koji se sastoje od više memorijski nezavisnih mašina) paralelizacija problema se zasniva na razmjeni poruka. Iako postoji mnogo načina za realizaciju ovog pristupa, u praktičnoj upotrebi u računarstvu visokih performansi je najkorišćeniji MPI (Message Passing Interface). Riječ je o specifikaciji biblioteke za prosljeđivanje poruka, a ne o konkretnoj implementaciji ili pristupu rješavanju problema. Iako je riječ samo o specifikaciji, u praksi se podrška za MPI sastoji i od biblioteke i od pomoćnih alata za kompajliranje izvornog koda, pokretanje izvršnog fajla, ali i skupa pomoćnih alata koji olakšavaju rad programeru.

MPI je podržan od strane širokog skupa programskih jezika (C, C++, Fortran, Java, itd) i operativnih sistema (Linux, Windows, Solaris, itd). Sam MPI je moguće koristiti u višeprocorskim mašinama, klasterima, ali i nezavisnim mrežno povezanim računarima (uz manja podešavanja). U osnovi svake slične biblioteke je implementirana minimalno podrška za slanje i primanje poruka, a MPI obezbjeđuje jednostavan i pouzdan rad sa porukama (poruka će ili biti isporučena ili će biti detektovano da nije isporučena). Naravno, svaka ozbiljna komunikaciona biblioteka podržava i blokirajuću i neblokirajuću komunikaciju, slanje poruka sa čvora na čvor, slanje poruka sa čvora na više čvorova, te kolektivnu komunikaciju u kojoj svi čvorovi šalju i primaju poruke sa svih čvorova (npr. u cilju sinhronizacije međurezultata nakon svakog koraka u kompleksnoj simulaciji). Kako MPI podržava veliki broj mogućnosti i pristupa rješavanju problema, očigledno je da nije riječ o jednostavnoj biblioteci. MPI u verziji 1 ima 128 funkcija/metoda, a u verziji 2 je taj broj narastao na 152. Ovako velik broj funkcija je očekivan jer za svaku jednostavnu funkcionalnost mora implementirati dva načina rada (blokirajući i ne), tri načina komunikacije (1:1, 1:N, M:N), dvije vrste parametara (fiksne i promjenjive dužine) što proizvodi minimalno 12 funkcija za svaku funkcionalnost.

Iako dostupna funkcionalnost može izgledati zastrašujuće, veliki broj programa je moguće realizovati upotrebom samo 6 osnovnih funkcija: inicijalizacija, dobijanje broja procesa, dobijanje broja mog procesa, slanje poruka, primanje poruka, kraj izvršavanja. Naravno, na ovaj način imamo malo da naučimo, ali i mnogo da programiramo. Ukoliko želimo da izbjegnemo bespotrebno programiranje i reimplementaciju funkcionalnosti koja je već implementirana, isplati se naučiti detaljnije MPI API i koristiti barijere (za sinhronizaciju), kolektivnu komunikaciju (koja automatski dijeli podatke na procese), obradu grešaka u komunikaciji, itd.

Vrlo jednostavan primjer MPI programa je dat u nastavku:

```
#include "mpi.h"
#include <stdio.h>

int main (int argc, char **argv) {
    int ukupno, cvor;

    MPI_Init (&argc, &argv);
    MPI_Comm_size (MPI_COMM_WORLD, &ukupno);
    MPI_Comm_rank (MPI_COMM_WORLD, &cvor);

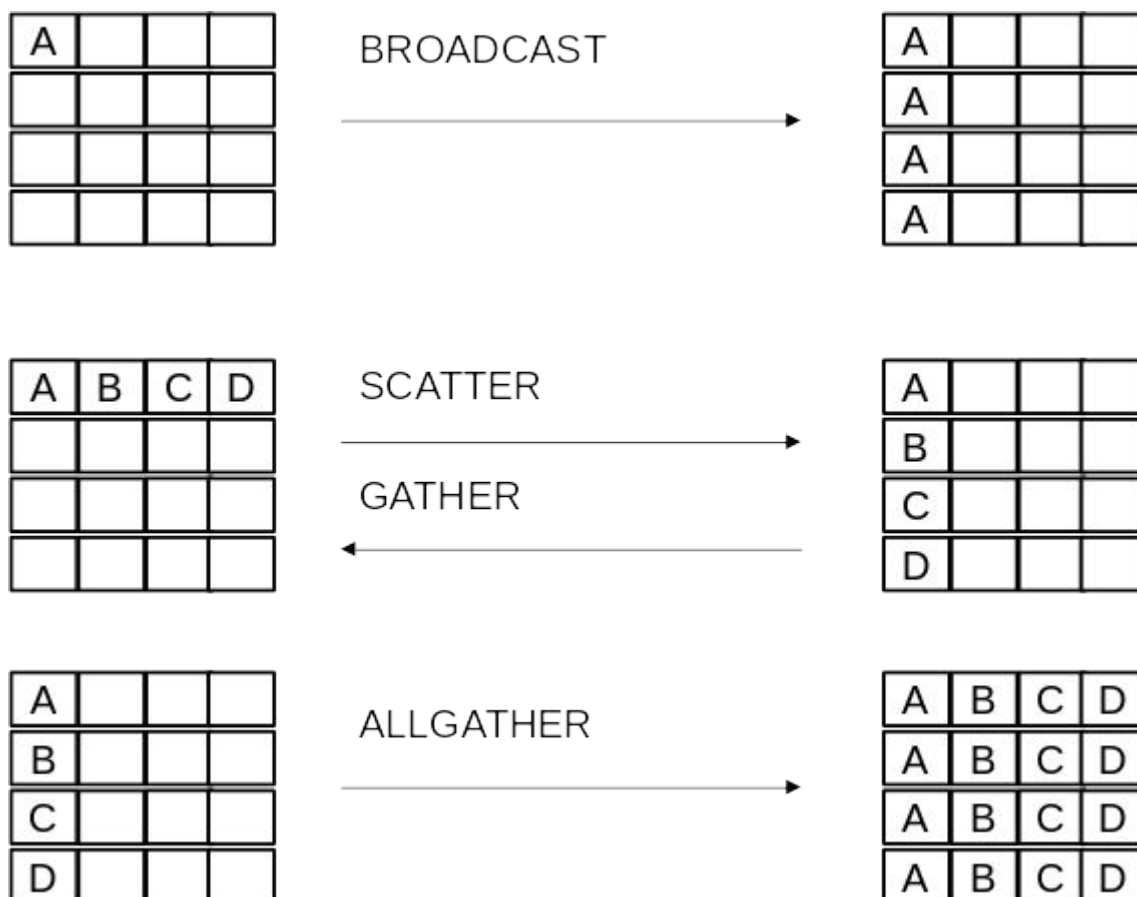
    printf ("Ukupno pokrenutih procesa: %d. Moj broj: %d\n", ukupno, cvor);

    MPI_Finalize ();
}
```

Da bi program kompajlirali, potrebno je prvo kod kompajlirati i za to koristimo poseban kompajler **mpicc** (koji interno koristi sistemski C kompajler), a samo pokretanje se razlikuje u odnosu na standardne programe. Naime, MPI programi se obično izvršavaju na više računara i nije praktično ručno pokretati procese na više računara i to na takav način da su pravilno iskonfigurisani (npr. da znaju na koji način i sa kojim drugim procesima komuniciraju). Kako je riječ o proceduri koju je moguće automatizovati, to je i učinjeno i za pokretanje MPI programa koristimo pomoćni alat **mpirun** koji minimalno prima broj procesa koje treba pokrenuti, ali i opcionu listu računara na kojima to treba i uraditi. Npr. moguće je sa računara comp0 pokrenuti 16 procesa na računarima comp1 do comp4, na svakom po četiri. Moguće je pokrenuti i različit broj procesa na svakom od računara, a sam MPI obezbjeđuje podršku za heterogeno izvršavanje (npr. dva računara su pod Linuxom, jedan Windows, a jedan je čak IBM big-endian server i drugačije interno predstavlja višebajtnu vrijednost - nema problema, MPI će korigovati razlike). MPI se obično koristi u velikim klasterima u kojima se samo pokretanje poslova i ne vrši ručno, makar i uz mpirun, već je koordinisano od strane sistema za upravljanje poslovima (JMS).

```
$ mpicc drugi.c -o drugi
$ mpirun -n 4 drugi
Ukupno pokrenutih procesa: 4. Moj broj: 0
Ukupno pokrenutih procesa: 4. Moj broj: 3
Ukupno pokrenutih procesa: 4. Moj broj: 1
Ukupno pokrenutih procesa: 4. Moj broj: 2
```

Na sljedećoj slici je prikazan primjer kolektivne komunikacije sa primjerima slanja istog podatka sa jednog čvora na sve čvorove (broadcast), raspodjele i prikupljanja niza podataka sa jednog čvora na sve čvorove (scatter i gather), te prikupljanja sa svih na sve čvorove (allgather).



15. Osnove UNIX filozofije

U doba nastanka UNIX operativnog sistema, resursi dostupni u računarskim sistemima su bili vrlo ograničeni, naročito prema savremenim shvatanjima, te je naglasak bio na što većem stepenu iskošćenja istih. Da bi u ranim računarskim sistemima to i postigli, upotreba asemblera je bila gotovo nezaobilazna, a s ciljem što boljih performansi, programi su uglavnom bili projektovani i realizovani u vidu kompleksnih monolitnih aplikacija koje bi obavljale sve neophodne operacije. Iako je na taj način ostvarivan glavni cilj, razvojem računarstva i kompleksnosti sistema, postalo je jasno da takav pristup slabo skalira i da proizvodi veliku količinu koda koji je vrlo kompleksan (težak za pravljenje, testiranje i održavanje) ali i teško ponovno upotrebljiv u za buduće projekte.

Osnova UNIX filozofije se može izreći kroz ideju razvoja softvera koji je **minimalistički i modularan**. Želimo da razvijamo kod koji će krasiti sljedeće osobine: jednostavan, jasan i kratak kod, koji je uz to i modularan, proširiv i ponovno upotrebljiv. Praktično sve što tada tradicionalni kod nije bio. Ova ideja se može posmatrati i kao iskazivanje frustracija stečenih razvojem velikih monolitnih aplikacija i ne predstavlja nešto što je usko vezano uz razvoj operativnih sistema ili sistemskih alata, nego je i danas vidljiv trend da se savremeni informacijski sistemi realizuju u vidu skupa slabo povezanih servisa (mikroservisna arhitektura), a ne kao veliki kompleksni sistem koji obavlja sve zadatke (monolitna arhitektura). Mnogi autori i poznati programeri su iznosili sopstveno viđenje filozofije razvoja softvera, a neka od najpoznatijih su:

Dennis Ritchie i Ken Thompson su formulisali sljedeće preporuke kojih su se držali u toku izrade UNIX sistema:

- Neka bude jednostavno pisati, testirati i izvršavati programe
 - Kod treba da bude jasan i razumljiv, bez pretjerivanja u upotrebi kompleksnih vidova kodiranja čak i kada to jezik dozvoljava. Bez obzira koliko se vi trudili da u C kodu neku funkcionalnost napišete upotrebom što "gušćeg" (ali i nerazumljivog) koda, nakon kompajliranja će biti izgubljene sve "fineze" kojima želite da se dokažete, ali će ostati sva zbunjenost radnih kolega koji će teško moći održavati i testirati takav kod.
- Potencirati interaktivnu upotrebu umjesto beč operacija
 - Uvijek pretpostavite da će programi biti korišćeni i od strane drugih vrsta korisnika i na drugačije načine, a ne samo u sklopu i načinu obrade kako ste zamislili u trenutku projektovanje. Sistem projektovan da radi interaktivno je mnogo lakše adaptirati u sistem koji ne zahtijeva interaktivni unos, nego obrnuto.
- Ograničenja resursa treba da diktiraju efikasan i elegantan kod
 - Kod ne treba bez potrebe proširivati i dodavati dijelove upitne koristi, prije dodavanja novih dijelova koda treba provjeriti da li je istu funkcionalnost moguće realizovati upotrebom postojećih funkcionalnosti.
- Radi kako propovijedaš - UNIX razvijati pod UNIX-om
 - Ako osobe koje razvijaju neki sistem moraju i da koriste taj sistem u razvoju, sami će i naići na veliki broj potencijalnih problema, ali i imati i direktno iskustvo upotrebe sistema, te na taj način biti dodatno motivisani da sam sistem bude što je moguće bolji.

S druge strane, Doug McIlroy je svoju filozofiju izrazio na sljedeći način:

- Pišite programe koji rade jednu stvar i rade je dobro (Do One Thing And Do It Well) DOTADIW
 - Umjesto da dodajete fundamentalno novu funkcionalnost u postojeći alat, napišite novi alat, na taj način kod ostaje minimalan, a novi alat će možda biti koristan i kasnije.
- Očekujte da izlaz bilo kog programa može biti ulaz nekog drugog programa
 - Logično, McIlroy je izmislio pipe |.
- Testirajte softver rano, ako ne radi - bacite ga
 - Uvijek je bolje da od početka uradite kako treba, nego da popravljate loš kod za koji je i veliko pitanje da li ga je uopšte moguće dovesti u željeno stanje.
- Koristite automatizaciju kad god možete, čak i ako mislite da je više nikada nećete tako koristiti
 - Bolje je nego da ručno obrađujete podatke jedan po jedan. Ovaj pristup omogućava kreiranje biblioteka korisnih funkcionalnosti što se može višestruko isplatiti u budućnosti, naročito ako radite u timovima (jer tada imate i pristup tuđim alatima)

Od brojnih ograničenja, preporuka i savjeta koji se navode u okviru UNIX filozofije, od kojih su neki i polu-šaljive prirode ("Cijela filozofija UNIX-a se svodi na to da se držite dalje od asemblera" Michale Sean Mahoney), vjerovatno je najvažnije da usvojite one koji i nemaju direktne veze sa razvojem operativnih sistema ili softvera, nego imaju opšti karakter i uvijek vrijede:

- DOTADIW - Do One Thing And Do It Well - McIlroy
- Snaga sistema nije u pojedinačnim alatima nego u njihovoj kombinaciji i saradnji - Brian Kernighan i Rob Pike

Na kraju se sve svodi na to da radite pametno, koristite prethodno stečena znanja i uvijek razmišljate **zašto** nešto radite, bez slijepog praćenja "modernih trendova" i upitnih preporuka.

16. UNIX i rad sa periferijama

Komunikaciju sa i upravljanje periferijama je moguće, na niskom nivou, izvesti na veći broj načina, od vrlo primitivnih pristupa na najnižem nivou, do upotrebe memorijskog mapiranja, prekida i direktnog pristupa memoriji od strane periferija sa ciljem ostvarivanja boljih performansi. Jednom kada riješimo te probleme na nivou operativnog sistema i drajvera, postavlja se pitanje kako da korisniku sistema omogućimo da tu funkcionalnosti i koristi. I u ovom slučaju je moguće više pristupa koji se, obično, svode na definisanje protokola za pristup periferiji, najčešće definisanjem *nekog* API-ja. Očigledno da postoje velike razlike između tastature i mikrofona, ali ih je moguće i posmatrati kao prilično slične - ulazni uređaji koji su nam dostupni dio po dio u vremenu (da ne kažemo karakter po karakter). S druge strane, sekundarna memorija i video kartica su takođe vrlo različiti, ali i tu je moguće povući određene paralele - možemo da im direktno pristupamo u blokovima (npr. dijelovima frejm bafera ili sektorima na HDD-u).

Kada prethodno navedena zapažanja o sličnosti povežemo sa idejama UNIX filozofije, da se zaključiti da je cilj napraviti jedan API, po mogućnosti neki koji već postoji i koji *nije potrebno mijenjati* za rad sa periferijama. Kako su periferije često praktično samo *posebni* blokovi memorije, ne bi bilo loše razmisliti o tome šta se još može koristiti na isti način.

Vrlo pojednostavljeno se ovaj pristup može opisati na sljedeći način: **Sve je fajl**. Kada kažemo da je sve fajl, to zaista i mislimo, te na isti način kao što radimo sa "običnim" fajlovima radimo i da diskovima, tastaturama, video karticama, serijskim portovima, itd. Sada nema potrebe da programer uči kako se pristupa paralelnom i serijskom portu, oba pristupa su ista i svode se na pristup specifičnom "fajlu", a posao operativnog sistema je da traženi pristup zaista i izvrši.

Primjeri rada sa fajlovima-periferijama pod Linux operativnim sistemom (pri čemu je Linux više-manje UNIX, ta rasprava je i kompleksna i često podložna pretjerano emotivnim stavovima, tako da ćemo je mi potpuno ignorisati) slijede, a za sada je interesantno primijetiti da se svi primjeri nalaze u **/dev/** direktorijumu:

- Serijskim portovima se pristupa putem fajlova `/dev/ttyS0` ili `/dev/ttySnešto`.
- Svi "diskovi" (sekundarna memorija) se takođe nalaze u `/dev/` i to `/dev/sda` kao **s** za SCSI (ili kompatibilan, SATA, USB, itd) **d** disk i to **a** prvi (b drugi, c treći, itd) u sistemu. Drugi disk na virtuelnom kontroleru (unutar virtuelne mašine) bi bio `/dev/vdb`. SSD u laptopu bi imao ime poput `/dev/nvme0n1` što znači da je priključen putem NVMe interfejsa i to na prvi interfejs, prvi imenski prostor (jer NVMe omogućava postojanje više) i to pristupamo trećoj particiji (npr. treća particija za prvi SATA disk bi bila `/dev/sda3`).
- Kako su sve periferije praktično preslikane u memoriju, a i obične fajlove takođe možemo preslikati u memoriju, bilo bi zanimljivo da i memoriji možemo da pristupamo kroz fajl. Ne samo da bi bilo zanimljivo nego je i moguće jer joj možete pristupiti (ako smijete) kroz fajlove `/dev/mem` (za svu fizičku memoriju računara) i `/dev/kmem` (za virtuelni adresni prostor kernela bez UI uređaja) ili `/dev/valkmem` (uključuje i UI). Dakle, ak oželite da promijenite sadržaj određenih lokacija u fizičkoj memoriji, sve što treba da uradite je da otvorite odgovarajući fajl i upišete podatke.
- Slučajni brojevi su na `/dev/urandom`, nule na `/dev/zero`, a "crna rupa" podataka na `/dev/null`.

17. Linux Filesystem Hierarchy Standard (FSH)

Linux je jezgro operativnog sistema otvorenog koda i sam po sebi ne nameće bilo kakva ograničenja po pitanju sadržaja ili organizacije fajl sistema. Teoretski je moguće imati i sistem koji se sastoji samo od kernela (uz određene modifikacije), ali je u praksi smislenije napraviti operativni sistem u kojem kernel izvršava *init* proces koji izvršava šta god da smo planirali (što znači da postoji samo jedan fajl u fajl sistemu). Ovakav pristup je čest u ugrađenim sistemima koji i jesu projektovani sa vrlo specifičnom namjenom u vidu. U opštem slučaju, funkcionalnost sistema nije tako usko ograničena i operativni sistemi zasnovani na Linux kernelu se distribuiraju u vidu distribucija (kombinacija Linux kernela, sistemskih i korisničkih alata, itd, najčešće uz podršku za upravljanje softverom realizovanom kroz neki od sistema za upravljanje paketima - package management system). Problem izazvan postojanjem više različitih distribucija je nešto sa čime se sreću programeri jer se njihove aplikacije mogu naći u vrlo različitim okruženjima, u zavisnosti od distribucije.

Kako je Linux vrlo popularan operativni sistem, a problem organizacije fajl sistema je problem koji u svakom slučaju moramo riješiti, Linux fondacija je na sebe preuzela rješavanje tog problema i kreirala standardnu hijerarhiju fajl sistema. Ukoliko distribucija poštuje propisanu organizaciju fajl sistema, aplikacije u sistemu se sada susreću sa jednim problemom manje, ali je i administratorima i korisnicima olakšano snalaženje u samom sistemu.

Organizaciju možemo podijeliti u tri grupe sa ciljem lakšeg shvatanja, ali uz razumijevanje da ova podjela postoji samo za potrebe ovog predmeta i ne predstavlja dio FSH. Osnovnu grupu direktorijuma čine:

- / root direktorijum fajl sistema
- /boot fajlovi neophodni za boot proces
- /bin osnovni izvršni fajlovi (alati)
- /sbin osnovni izvršni sistemski fajlovi (alati)
- /etc tekstualni konfiguracioni fajlovi
- /lib biblioteke (i /lib32 i /lib64)

Od interesa za korisnike i servise namijenjene za upotrebu od strane korisnika su:

- /home korisnički direktorijumi (sadrži korisničke naloge npr. /home/student)
- /root korisnički direktorijum korisnika root (olakšava rad i otklanjanje problema, jer /home nije neophodan za funkcionisanje sistema, a korisnik root gotovo uvijek postoji u nekoj formi)
- /tmp privremeni fajlovi (ne mora biti na disku)
- /usr sekundarna hijerarhija read-only za korisničke programe (u idealnom slučaju sadrži bin, sbin, lib, itd. direktorijume)
- /var promjenjivi fajlovi (podaci sistemskih aplikacija, logovi, ...), na primjer, često se fajlovi koji čine web prezentaciju koju servira web server nalaze u /var/www, a fajlovi koji predstavljaju MySQL bazu podataka u /var/lib/mysql

- /srv prostor predviđen za fajlove koji koriste serveri (web, FTP, itd) često su u praksi u /var direktorijumu, kao što je ranije navedeno
- /opt korisnički programi - samodovoljni paketi koji sadrže sve što im treba, uključujući biblioteke i pomoćne programe koji bi inače bili instalirani na nivou sistema i raspoređeni u druge direktorijume, ali često kod distribuiranja komercijalnog softvera to nije izvodivo rješenje pa se koristi /opt (takođe olakšava uklanjanje softvera jednostavnim brisanjem direktorijuma)

Direktorijumi posebne namjene su:

- /dev uređaji (devices)
- /media uklonjivi/privremeni uređaji (DVD, USB memorija, itd)
- /mnt dugoročno priključeni fajl sistemi (mrežni, diskovi, itd)
- /run trenutne informacije o sistemu
- /sys informacije o uređajima, drajverima, itd.
- /proc informacije o procesima, kernelu, itd.

Naročito je interesantan /proc direktorijum koji, između ostalog, sadrži i po direktorijum za svaki proces koji postoji u sistemu (init ima broj 1) tako da komandnu liniju kojom je pokrenut neki proces možemo pronaći jednostavnom upotrebom *cat* alata, npr. za proces broj 143252:

```
$ cat /proc/143252/cmdline
watch -n 10 df -h
```

Direktorijum /proc sadrži mnoge zanimljive informacije o sistemu, na primjer:

- cat /proc/cpuinfo detaljne informacije o procesorima u sistemu
- cat /proc/meminfo detaljne informacije o memoriji u sistemu
- cat /proc/vmstat statistike vezane za virtuelnu memoriju
- cat /proc/iomem memorijski mapiran UI (neophodan root pristup)
- cat /proc/ioports UI portovi za sve uređaje (neophodan root pristup)
- cat /proc/interrupts pregled prekida po CPU jezgrima i uređajima

Navedeni direktorijumi ne sadrže "stvarne" fajlove, njihovim sadržajem upravljaju virtuelni fajl sistemi (procfs, sysfs, tmpfs, itd) koji omogućavaju pristup brojnim internim atributima operativnog sistema, na primjer:

```
$ mount
sysfs on /sys type sysfs (rw,nosuid,nodev,noexec,relatime)
proc on /proc type proc (rw,nosuid,nodev,noexec,relatime)
udev on /dev type devtmpfs (rw,nosuid,noexec,relatime,mode=755, ...)
devpts on /dev/pts type devpts (rw,nosuid,noexec,relatime, ...)
tmpfs on /run type tmpfs (rw,nosuid,nodev,noexec,relatime, ...)
...
```

I na ovom primjeru možemo da vidimo da su osnovne ideje UNIX-a i danas prisutne u morenim operativnim sistemima i da svima olakšavaju život i rad.

18. Osnove kontrole pristupa

Kontrola pristupa je vrlo široko polje, čak i samo u računarskom okruženju. Iako mi često posmatramo samo softverske aspekte (na koje ćemo se i u ovom slučaju ograničiti), u stvarnom svijetu ne možemo jednostavno ignorisati probleme fizičke sigurnosti, pa čak ni zaštite od poplave ili drugih prirodnih nepogoda.

U okviru ovog predmeta, mi ćemo se ograničiti na sljedeće probleme:

- autentifikacije - potvrđivanje identiteta korisnika ili uopšteno neke tvrdnje,
- autorizacije - funkcija definisanja politike pristupa, te
- provođenje politike pristupa - (ne)dozvoljavanje pristupa.

U praksi posmatramo proces koji na izlazu dozvoljava ili ne dozvoljava izvršavanje određenog upita, što može obuhvatati i pristup resursima, vršenje radnji, itd. Ovaj proces može objedinjavati i autentifikaciju i autorizaciju i provođenje politike pristupa u jednom procesu, ali je moguće i da se radi od dva, tri ili više odvojenih procesa koji su povezani na određeni način sa ciljem zajedničkog funkcionisanja (UNIX filozofija).

Kada govorimo o autentifikaciji korisnika u računarskim sistemima, faktore koji se koriste pri autentifikaciji možemo klasifikovati u tri vrste:

- Nešto što znamo (znanje)
 - Predstavlja informaciju koju čuvamo u našem pamćenju: šifra, PIN, odgovor na pitanje, itd. Trenutno nije moguća direktna nevoljna ekstrakcija ove vrste podataka (ako ne računamo prijetnju kaznom ili drugi vid uslovljavanja).
- Nešto što imamo (posjedovanje)
 - Predstavlja objekat koji posjedujemo: ID kartica, token, uređaj, softver, itd. Apsolutno ne mora da se radi o fizičkom objektu, moguće je, na primjer, da je riječ o softveru koji se izvršava na našem telefonu i predstavlja faktor autentifikacije. Otudjenje, zakonsko oduzimanje ili slučajni gubitak objekta je moguć i tome moramo voditi računa.
- Nešto što jesmo (biometrija)
 - Neki od naših fizičkih atributa koji nije jednostavno ponovljiv u populaciji: otisak prsta, mrežnjača, izgled lica, glas, itd. Potrebno je da je atribut dovoljno jedinstven (broj prsta na ruci nije), ali i dovoljno stabilan (masa u nanogramima je vjerovatno jedinstvena, ali teško da je konstanta u toku života ili dana).

Izuzetno je važno shvatiti da vrste faktora nisu "bolje" ili "gore" jedna od druge, one su **različite** i u tome je njihov smisao. Na primjer, šifru je teško neprimjetno izvući iz nas, ID karticu je relativno lako ukrasti, ali sama kartica je mnogo teža za kopiranje ili pogađanje (jer može sadržavati, npr. 65536-bitni ključ) u odnosu na šifru (koja mora biti dovoljno jednostavna da je zapamtimo - recimo 60-tak bita). Karticu, takođe, možemo relativno lako da proglasimo nevažećom i izdamo novu, šifru možemo da promijenimo, ali sa otiskom prsta nemamo sreće - ako neko nađe način da ga vjerodostojno reprodukuje mi i nismo u situaciji da ga naprosto promijenimo na našim rukama.

Pri procesu autentifikacije se često susrećemo sa problemom koliko faktora koristimo u autentifikaciji. Šta dobijamo, a šta gubimo upotrebom više faktora i kakvih? Upotreba više faktora iz iste vrste i nije veliki napredak jer je jednako lako ukrasti i ID karticu i token, kao i ukrasti ili pogoditi šifru i PIN. Kada se govori o višefaktorskoj autentifikaciji, obično se podrazumijeva da se radi o više faktora iz različitih vrsta faktora.

Najjednostavnija i najmanje sigurna, ali i, makar danas, najčešća je jednofaktorska autentifikacija. Najčešće je realizovana u vidu tajne pristupne lozinke (šifre) ili PIN broja i koristi se za sisteme u kojima nivo zaštite nije od kritične važnosti (ulazak u stubište zgrade, lični računari, itd).

Viši nivo zaštite pruža višefaktorska autentifikacija koja koristi dva ili više faktora iz različitih vrsta. Danas su u upotrebi široko prisutne brojne kombinacije, na primjer: kartica i PIN za finansijske transakcije, šifra i token za pristup prostorijama, otisak prsta i šifra i token za još viši nivo sigurnosti (ali i savremeni telefoni sa čitačem otiska prsta ili prepoznavanjem lica koji imaju instaliran i neki authenticator softver koji, uz pristupnu šifru ili PIN, koristimo za neki servis).

Jedna od velikih mana klasičnih sistema za autentifikaciju leži u činjenici da se sama autentifikacija vrši u trenutku prijave na sistem i, bez obzira na to koliko je napredna autentifikacija na, na primjer, našem telefonu (koristimo i otisak i PIN), ukoliko napadač ukrade uređaj dok ga mi koristimo - autentifikacija nam neće pomoći (jer je već završena). Da bi riješili ovaj i slične probleme, postoje sistemi koji kontinualno autentifikuju korisnika (npr. nadgledanjem aktivnosti ili biometrije korisnika i detekcijom uljeza). Osim detektovanja otuđenja pristupa, ovi sistemi često nude i mogućnost da detektuju neuobičajeno ponašanje uobičajenih korisnika sistema (npr. nezadovoljni zaposleni koji kopira veliku količinu poslovne dokumentacije *nakon* korektne autentifikacije).

Autorizacija se često miješa sa problemom provođenja politike pristupa. Usko posmatrano, autorizacija je samo funkcija *definisanja* politika pristupa, dok je provođenje prava pristupa ono što se često podrazumijeva pod autorizacijom. Na slici je prikazan slučaj dva stanja sistema, prvi predstavlja autorizovano stanje, a drugi neautorizovano jer sadrži unos koji nije dozvoljen politikom pristupa (korisnik "Miki" nema RWX pristup fajlu "Fajl 3"). Samo provođenje prava pristupa treba da obezbijedi da se sistem ne može naći u neautorizovanom stanju, najčešće na taj način što će onemogućiti izvršavanje određenog upita. Zamislimo fajl sistem koji je realizovan tako da nam ne prikazuje imena fajlova kojima nemamo pristup (*security through obscurity*), ali ako možemo da pogodimo ime fajla - dobićemo i pristup (kao na slici). Ovo je loše rješenje i ne implementira politiku pristupa na korektan način jer sistem može vrlo lako doći u neautorizovano stanje (primjer na slici).

Matrica pristupa (protection matrix)

Autorizovano stanje				Neautorizovano stanje			
	Fajl 1	Fajl 2	Fajl 3		Fajl 1	Fajl 2	Fajl 3
Miki	RWX	R		Miki	RWX	R	RWX
Mini	RX	RWX	RX	Mini	RX	RWX	RX
Pajo			RWX	Pajo			RWX

19. Modeli kontrole pristupa

Postoji veliki broj načina da definišemo i provodimo politike pristupa, ali prije sve je neophodno definisati ko ima pravo da definiše politike pristupa i za koje objekte. Obično je najvažnije pitanje koje, u velikoj mjeri, određuje i odabir konkretnog modela kontrole pristupa je da li korisnik ima pravo da upravlja objektima koje kreira ili ne.

Obično se kao primjer navode fajl sistemi, prije svega jer su nam najbliži i sa njima tradicionalno imamo najviše iskustva, ali je danas, porastom popularnosti društvenih mreža i drugih internet servisa u kojima korisnici generišu sadržaj ovaj problem poprimio potpuno nove dimenzije (uključujući probleme vezane za aspekt privatnosti).

U okviru ovog predmeta, mi ćemo obraditi četiri osnovna modela kontrole pristupa:

- Discretionary access control (DAC)
 - U ovom modelu korisnik ima puno (diskreciono) pravo raspolaganja objektima čiji je vlasnik i sistem se ne upliće u taj proces nego se ograničava na provođenje politika koje su definisali korisnici za svoje objekte.
- Mandatory access control (MAC)
 - U ovom modelu korisnici nemaju veliki stepen slobode, naprotiv, sistem je taj koji propisuje pravila i politike pristupa, te ih i provodi. Obično se koriste u sistemima sa višim stepenom zaštite i strogo propisanim procedurama (vojska, zdravstvo, zakonodavstvo, itd).
- Role-based access control (RBAC)
 - Prava pristupa i definisanja politika zavise od uloge (*role*) koju u datom trenutku obavlja posmatrani korisnik u sistemu. Može da emulira i DAC i MAC po potrebi.
- Attribute-based access control (ABAC)
 - Predstavlja novu generaciju modela kontrole pristupa i nudi izuzetno visok nivo fleksibilnosti, ali po cijenu povećanja i kompleksnosti implementacije sistema kontrole pristupa.

Svaki od navedenih modela ima određene prednosti i nedostatke, ali je vjerovatno korektnije reći da za svaki postoji slučaj upotrebe u kojem je bolji izbor u odnosu na druge modele. Na primjer, iako je ABAC toliko fleksibilan da može, bez ikakvih ograničenja, da emulira sve druge modele, on je, takođe, potencijalno toliko kompleksan da je potpuno neupotrebljiv u sistemima u kojima postoji vrlo ograničena količina procesorskih ili memorijskih resursa ili su vrhunske performanse od kritične važnosti.

Posao inženjera često upravo i jeste da bude svjestan široke lepeze mogućih rješenja i odabir odgovarajućeg rješenja za konkretan problem koji analizira. Pravilan odabir pristupa neće samo riješiti taj problem, nego će omogućiti i razvoj i rast sistema u budućnosti, često i na nepredvidiv način. Sve navedeno nas često usmjerava na stranu odabira modularnih rješenja pri čemu se ponovo vraćamo na osnovne ideje razvoja UNIX operativnog sistema.

20. Modeli kontrole pristupa - DAC

Kao što je prethodno navedeno, u DAC (Discretionary Access Control) modelu kontrole pristupa korisnik, koji može da predstavlja i vlasnika objekta, ima potpunu slobodu upravljanja politikom pristupa samom objektu. Potpuno diskrecioni sistemi su oni u kojima ne postoje nikakva ograničenja definisana od strane sistema (nema MAC komponente) iako većina sistema predstavlja mješoviti pristup, s tim što je, u ovom slučaju, dio koji je pod kontrolom korisnika dominantan. U osnovi postoje dva pristupa pri realizaciji DAC pristupa:

- Owner based - zasnovan na korisničkom vlasništvu nad objektom. Vlasnik ima punu slobodu da definiše politiku pristupa objektu, uključujući i prenos vlasništva na drugog korisnika koji time stiče potpunu kontrolu nad objektom, što opet uključuje i prenos vlasništva na trećeg korisnika ili vraćanje na originalnog vlasnika.
- Capabilities based - zasnovan na ideji da nije neophodno identifikovati ni korisnika objekta ni vlasnika istog, već da je dovoljno da korisnik bude entitet koji posjeduje *ključ* (ili token) koji mu omogućava da slobodno pristupa nekom objektu. Taj isti ključ može podijeliti sa trećim stranama i na taj način im omogućiti pristup kontrolisanom objektu bez potrebe za utvrđivanjem vlasništva ili identiteta.

DAC model se klasično vezuje za primjere višekorisničkih operativnih sistema i fajl sistema koji podržavaju takav rad (ext2, ext3, ext4, NTFS, xfs, jfs, itd). U ovom primjeru korisnik koji kreira fajl postaje njegov vlasnik i ima punu kontrolu nad njim. Osim što može da sam fajl izvršava, čita, mijenja ili uništava, može dodijeliti i određena prava pristupa i sa drugim korisnicima ili korisničkim grupama, kao i sa svim korisnicima sistema. Vlasnik sve vrijeme ima potpunu kontrolu i definiše ko ima pravo pristupa fajlu, a samo zato što neki drugi korisnik ima pravo izvršavanja navedenog fajla ne znači da to isto pravo može proslijediti trećem korisniku.

U novije vrijeme je ovaj način upravljanja pristupom objektima u upotrebi u internet baziranim sistemima za rad sa dokumentima, društvenim mrežama, itd. Korisnik koji generiše dokument u okviru takvog servisa isti može podijeliti sa drugim korisnicima sistema i dozvoliti im pristup na željeni način što predstavlja *owner* bazirani način rada sistema. Međutim, ukoliko je korisnik objekat podijelio putem linka (koji obično sadrži i token za pristup objektu), primalac tog linka obično može pristupiti objektu i bez prijavljivanja na sistem (kao eksterni anonimni korisnik), ali i link podijeliti sa trećim stranama bez dozvole vlasnika objekta (koji u tom slučaju i nije više važan, sve dok ne povuče dijeljenje putem linka) što je demonstracija rada sistema zasnovanog na *capabilities* pristupu.

Još jedan primjer koji nije vezan za korisnike sistema, te time ilustruje da nije riječ samo o "korisnicima" u užem smislu riječi je i dijeljenje memorije između procesa. U okviru operativnog sistema proces koji se izvršava sam definiše da li želi da dijeli neki dio memorije sa drugim procesom i upravlja tim dijeljenjem.

Očigledno je da nije ista kompleksnost jednostavnog označavanja dijela memorije za dijeljenje i dijeljenja korisničkih internet resursa, ali moramo da vodimo računa i o tome koliko često se vrši provjera prava pristupa i kakve su posljedice. Ukoliko bi provjera prava pristupa memoriji trajala 1 ms svaki put kada pristupamo memoriji, to bi učinilo većinu aplikacija neupotrebljivim, dok bi u slučaju internet baziranog dijeljenja kašnjenje bilo gotovo neprimjetno.

21. Modeli kontrole pristupa - MAC

Kod MAC (Mandatory Access Control) modela kontrole pristupa sistem definiše obavezna prava pristupa i ne ostavlja korisnicima ili vlasnicima mnogo slobode. U čistim MAC sistemima korisnici (vlasnici) nemaju nikakvu slobodu defisanja politika pristupa jer su sve politike definisane od strane sistema.

Istorijski posmatrano, MAC sistemi su prvu primjenu pronašli u vojnim i drugim osjetljivim sistemima čime je ostvarena veza termina MAC i visoko pouzdanih i robusnih sistema koji su projektovani na takav način da su u stanju da izađu na kraj sa svim predvidivim napadima, uz strategiju da uvijek griješe na stranu sigurnosti. Iako se ovaj model koristi i u civilne svrhe (zdravstvo - strogo je propisano ko i na koji način ima pravo pristupa podacima pacijenata; finansijski sektor - strogo je propisano ko i na koji način može da izvršava transakcije ili upite; itd) upotreba u vojnom okruženju se i danas posmatra kao "zlatni standard" i primjeri vezani za MAC su obično vezani za vojnu upotrebu (postojanje jasne hijerarhije i precizno definisanih procedura u vojsci olakšavaju objašnjavanje primjera). Često se uz MAC koristi i pojam MLS (Multilevel Security - višenivovska sigurnost) koja, obično, upotrebom labela omogućava kreiranje više nivoa u hijerarhiji prava pristupa strogo vodeći računa da ne dođe do narušavanja hijerarhije.

Kada govorimo o operativnim sistemima i MAC modelu, primjera ima mnogo, od činjenice da većina operativnih sistema koji izgledaju kao da koriste DAC model posjeduju i MAC komponentu, preko toga da proces koji se izvršava ne može sam sebi dodijeliti pravo da koristi neki sistemski resurs ukoliko mu to pravo nije dodijeljeno od strane sistema, do upotrebe procesorskih prstenova ili jednostavnog privilegovan-neprivilegovan pristupa.

Ukoliko je potrebno da operativni sistem podržava MAC model kontrole pristupa, moguće je ili odabrati neki namjenski pisan operativni sistem ili operativni sistem opšte namjene koji implementira podršku za MAC. Neki od primjera drugog slučaja su:

- Microsoft Windows Server 2008 ili noviji - MLS pristup zasnovan na *Integrity Levels* pristupu (svi procesi i resursi imaju odgovarajući nivo integriteta i nije moguće narušavanje te hijerarhije - za svaki pristup imenovanom objektu se vrši provjera)
- FreeBSD 7.2 ili noviji kroz TrustedBSD projekat podržava i MLS.
- Linux - kao i inače, Linux ne definiše jedno rješenje nego omogućava razvoj sigurnosnih proširenja kernela upotrebom LSM (Linux Security Modules) interfejsa kroz koji je moguće koristiti:
 - SELinux (Security Enhanced Linux) razvijen od strane NSA (National Security Agency) koji omogućava kontrolu nad svim ili samo željenim resursima, a podržava i MLS.
 - AppArmor omogućava nadzor i kontrolu nad definisanim aplikacijama i koristi se u slučaju da ne želimo da realizujemo MAC model na nivou kompletnog sistema.
 - Smack (Simplified Mandatory Access Control Kernel) omogućava pojednostavljenu upotrebu MAC modela i često se koristi u ugrađenim sistemima. TOMOYO Linux je još jedna alternativa za ugrađene sisteme specifična po tome što je sistem moguće pokrenuti u režimu u kom je sve dozvoljeno i svi pristupi se bilježe da bi na osnovu toga mogli definisati politike pristupa.

22. Modeli kontrole pristupa - RBAC

Vrlo često se nađemo u situaciji da ni DAC ni MAC ne ispunjavaju potrebe sistema koji projektujemo. Čak i upotreba hibrida opisanih u prethodnom tekstu (DAC sistem sa MAC komponentom) neće biti dovoljno fleksibilno rješenje. U takvim slučajevima, kao i kada nam je potreban pristup koji će omogućiti da se sistem razvija i mijenja u budućnosti, često koristimo RBAC (Role-based Access Control) model kontrole pristupa. Ovaj model je neutralan po pitanju politike pristupa što mu omogućava da emulira i MAC i DAC po potrebi, ali i da bude ono što oni ne mogu biti.

Zasniva se na ulogama (*roles*) i pravima pristupa (*permissions*), te korisnicima (*users*) kojima ona mogu biti dodijeljena. Sama prava nisu ograničena na puki pristup nekom objektu i mogu biti mnogo preciznije definisana (npr. za neki fajl nije dozvoljeno ili zabranjeno upisivanje podataka, dozvoljeno je *dodavanje* podataka ili inkrementovanje nekog podatka ili ovjera semestra za studenta). Prava pristupa se obično ne dodjeljuju direktno korisniku nego ulogama u sistemu, a korisnici mogu imati određene uloge u datom trenutku. Na taj način je olakšan problem administracije u sistemima sa velikim brojem prava i korisnika jer ne dolazi do eksplozije mogućih kombinacija.

Sam RBAC se zasniva na tri jednostavna pravila:

1. Subjekt može koristiti pravo samo ako mu je dodijeljena uloga
2. Trenutna uloga subjekta mora biti autorizovana
3. Subjekt može koristiti pravo samo ako je to pravo autorizovano za trenutnu ulogu

Pojednostavljeno rečeno: subjekt može koristiti pravo samo ako je autorizovan za njega (sistem ne može biti u neautorizovanom stanju).

Još jedan od načina upravljanja potencijalno velikim brojem prava i uloga u sistemu je upotreba hijerarhijskih RBAC sistema. Na primjer, neka demonstrator za laboratorijske vježbe ima pravo da održava evidenciju prisustva na vježbama u nekom hipotetičkom informacionom sistemu univerziteta. Asistent ima pravo uradi sve što i demonstrator, ali i da unese bodove za prisustvo na vježbama. Nastavnik ima pravo da uradi sve što i demonstrator i asistent, ali i da unese konačni broj bodova i zaključi ocjenu. Bez hijerarhijskog pristupa bi postojao relativno velik broj veza između prava i uloga (demonstrator: prisustvo; asistent: prisustvo, bodovi za prisustvo; nastavnik: prisustvo, bodovi za prisustvo, konačni bodovi, ocjena), ali i velika mogućnost greške pri dodavanju novog prava za demonstratora jer je neophodno da sada to pravo dodijelimo i asistentu i nastavniku. Očigledno je da u kompleksnim sistemima održavanje politika postaje vrlo kompleksno. Međutim, ukoliko uvedemo hijerarhiju po kojoj nastavnik ima sva prava asistenta koji ima sva prava demonstratora, sada dodjela postaje jednostavna (demonstrator: prisustvo; asistent: bodovi za prisustvo; nastavnik: konačni bodovi, ocjena), mogućnost greške je smanjena, a buduće održavanje pojednostavljeno. Ovaj pristup ne podrazumijeva postojanje jednog korijena hijerarhije i sama hijerarhija više odgovara nasljeđivanju u objektno-orijentisanoj paradigmi nego fajl sistemu. Ovaj pristup se izuzetno često koristi u razvoju višekorisničkih web aplikacija i većina ozbiljnih razvojnih okvira ima kvalitetno realizovanu podršku za RBAC model.

23. Modeli kontrole pristupa - ABAC

ABAC (Attribute-based Access Control) je takođe poznat i kao PBAC (Policy-based Access Control) ili, u Microsoft okruženju, kao CBAC (Claims-based Access Control). Omogućava *izuzetno* visok nivo fleksibilnosti u definisanju politika pristupa jer omogućava upotrebu formula/algoritama koji sadrže IF-THEN-ELSE i potencijalno kompleksne logičke iskaze.

Sam model je zasnovan na postojanju atributa, koji mogu biti praktično bilo šta i dijele se u četiri osnovne grupe:

- Atributi subjekta - opisuju korisnika ili entitet koji želi da izvrši akciju nad objektom. Primjeri: korisničko ime i grupa, ime i prezime, radno mjesto, IP adresa, itd.
- Atributi akcije - opisuju akciju koju subjekat želi da izvrši. Primjeri: čitanje, pisanje, kreiranje, zaključivanje ocjene, ovjera semestra, itd.
- Atributi objekta - opisuju objekat nad kojim subjekat želi da izvrši akciju. Primjeri: status zapisa, broj bodova, vrijeme i datum, itd.
- Atributi konteksta - atributi koji opisuju kontekst (okruženje) u kojem se akcija izvršava. Primjeri: trenutni datum i vrijeme, broj aktivnih instanci servisa, režim rada servisa, itd.

Upotreba ABAC modela nam omogućavamo da definišemo pristup na takav način da nekom fajlu imate pun pristup (čitanje i pisanje) samo za vrijeme radnog vremena ako pristupate iz lokalne mreže, samo čitanje ako pristupate za vrijeme radnog vremena od kuće ili iz lokalne mreže van radnog vremena, a u ostalim slučajevima pristup nije dozvoljen. Takođe, za neki objekat imate pravo izmjene dokumenta samo iz lokalne mreže dok je statusu "u izradi", samo čitanje iz lokalne mreže kad je u statusu "prijedlog" i čitanje sa svih adresa ako je u statusu "objavljen", itd.

Vidljivo je da je fleksibilnost izuzetna i da je moguće vrlo precizno definisati prava pristupa. Očigledno je i da je i kompleksnost velika, a preporučena arhitektura ABAC sistema se sastoji od:

- PEP (Policy Enforcement Point) - tačka u sistemu koja je zadužena za zaštitu pristupa objektima kojima želimo da ograničimo pristup. PEP analizira svaki zahtjev i pravi zahtjev za autorizaciju koji prosljeđuje do PDP tačke.
- PDP (Policy Decision Point) - tačka u kojoj se analizira pristigli zahtjev i na osnovu definisanih politika pristupa isti odobrava ili odbija. PDP može po potrebi da dobavi dodatne informacije (attribute) posredstvom PIP tačaka.
- PIP (Policy Information Point) - omogućava vezu između PDP i eksternih izvora podataka, kao što su sistemi zaduženi za upravljanje identitetom korisnika, eksterne baze podataka, sistemi za nadzor, sistemi za detekciju neuobičajenog ponašanja, itd.

Primijetimo da se ova ABAC arhitektura ne bavi direktno problemom upravljanja atributima.

ABAC za razliku od prethodno opisanih modela (DAC, MAC i RBAC) nije fokusiran samo na korisnika nego je u stanju i da iskoristi dodatne izvore informacija. Iako je ova tvrdnja tehnički korektna, vrijedi napomenuti da veliki broj implementacija RBAC modela podržava i princip rada vrlo blizak ABAC modelu, čak do nivoa da se dozvola pristupa može realizovati kao funkcija u programskom jeziku razvojnog okvira što omogućava implementaciju bilo kog željenog pristupa.